Flink SQL reports

Seven deterministic reports (plus an optional eighth, AI-generated one — see §2.0 [OPTIONAL] AI Root-Cause Analysis (RCA)) that read two streams of isotope metadata — the **produce side** (the `x-isotope-*` headers stamped on every event topic record by `IsotopeProducerInterceptor`) and the **consume side** (the value-less marker records on `isotope_consume_edge_markers` emitted by `IsotopeContext.recordConsume`) — and surface what's flowing where, how fast, and how reliably. Each report runs as a long-lived `INSERT INTO <report>_1m SELECT …` streaming job. The aggregation logic is identical across runtimes; only the source/sink DDL and function-registration glue differ.

Riding alongside them is one **collector** INSERT — not a report, and not an aggregation. Where the seven reports *interpret* isotope headers that an interceptor already wrote, the collector *writes* them: it forwards `orders.placed` to `orders.flink_enriched` 1:1, appending a Flink hop so Flink appears in the topology graph as a producer rather than only a reader. Both runtimes run all eight INSERTs as a single job. See docs/flink-collector.md.

The headline addition is the `bipartite_topology` report: it unions the produce and consume views to render the pipeline as a literal **bipartite graph** from graph theory — services in one vertex set, topics in the other, edges crossing between them in both directions. See root README §1.0 "How an Isotope Traverses an Event Pipeline" for the full motivation.

Runtime	Reports	Sink format	Where the SQL lives
Confluent Platform Flink (Flink 2.1 CMF Application on minikube)	7 (latency, topology, bipartite-topology, hop-distribution, coverage, stuck-trace, latency-percentiles) + 1 collector	`avro-confluent` (SR-framed Avro)	scripts/flink/sql/cp/ — run by `IsotopeReportsJob` as one `StatementSet`, deployed by scripts/deploy-cmf-flink-reports.sh
Confluent Cloud for Apache Flink (CCAF)	7 (same set as CP) + 1 collector	`proto-registry` (SR-framed Protobuf)	Inlined as `confluent_flink_statement` resources in terraform/setup-ccaf.tf, the INSERTs consolidated into one `EXECUTE STATEMENT SET` — applied by scripts/deploy-cc-flink-reports.sh

Control Center deserializes both sink formats natively.

Table of Contents

- **1.0 What each report computes**
- **2.0 [OPTIONAL] AI Root-Cause Analysis (RCA)**
- **3.0 Layout**
- **4.0 Wire-format detail (CP only)**

- 5.0 PTF JAR
- 6.0 Operations
  - 6.1 CP Flink on minikube
  - 6.2 CCAF

1.0 What each report computes

Every report aggregates over a `TUMBLE(event_time, INTERVAL '1' MINUTE)` window. The "Source view" column names the typed view (or views) the report reads from; the views in turn filter the single `isotope_raw` source table by the presence/absence of the `x-isotope-consumer-service` header.

Every report also carries a **`pipeline`** column (decoded from the `x-isotope-pipeline` header) as a leading grouping dimension, so rows for an `orders` pipeline and a `location` pipeline never aggregate together. The "What it computes" column below lists the *additional* keys each report groups by, on top of `pipeline`.

Report	Source view	What it computes	Runtimes
`latency`	`isotope`	avg / min / max end-to-end latency by `origin_service × current_topic`	CP, CCAF
`topology`	`isotope`	produce-edge counts per `(producer_service, topic)` per minute	CP, CCAF
`bipartite_topology`	`isotope` and `consume_events`	full bipartite graph: produce edges plus consume edges per minute	CP, CCAF
`hop_distribution`	`isotope`	record counts bucketed by `hop_count` per topic per minute	CP, CCAF
`coverage`	`isotope`	distinct traces per topic per minute	CP, CCAF
`stuck_trace`	`isotope`	alerts (via `STUCK_TRACE_PTF`) for traces idle ≥60s of event time before reaching `orders.fulfilled`	CP, CCAF
`latency_percentiles`	`isotope`	p50 / p95 / p99 (via `LATENCY_PERCENTILES` PTF, T-Digest)	CP, CCAF

Plus one non-report INSERT. `75_flink_collector.fql` is not an aggregation and has no window — it forwards `orders.placed` to `orders.flink_enriched` 1:1, appending a Flink hop via the `ISOTOPE_APPEND_HOP` UDF. It rides in the same job as the seven reports on both runtimes. See docs/flink-collector.md.

Format-by-runtime (not-by-domain). CP's reports land on **Avro+SR** (`'value.format' = 'avro-confluent'` in scripts/flink/sql/cp/05_report_sinks.fql). CCAF's reports land on **Protobuf+SR** (`'value.format' = 'proto-registry'` in each sink's `WITH` clause in terraform/setup-ccaf.tf). The

two runtimes' SQL is otherwise unshared: CP's lives hardcoded in scripts/flink/sql/cp/, CCAF's lives inline as `confluent_flink_statement` resources in terraform/setup-ccaf.tf.

Why `latency_percentiles` is a `ProcessTableFunction` (PTF). Since CCAF does not support User-Defined AGGregate functions (UDAGG), I implemented the percentiles report as a `ProcessTableFunction` to keep it portable across runtimes. `LATENCY_PERCENTILES` (class `LatencyPercentilesPTF`) performs its own 1-minute tumbling-window aggregation over a T-Digest sketch using per-window state and event-time timers. **A PTF avoids that UDAGG restriction, so it registers and runs on both runtimes, just like `STUCK_TRACE_PTF`. Both runtimes therefore run the same seven reports:** `latency` (avg/min/max), `topology` (produce-side), `bipartite_topology` (full service↔topic↔service graph), `hop_distribution`, `coverage`, `stuck_trace`, and `latency_percentiles` (p50/p95/p99).

The demo *event* topics (`orders.placed`, `orders.enriched`, `orders.fulfilled`) ride Protobuf+SR via the Java app's `DemoEvent` schema on both runtimes — those are written by the Kafka producer client, not by Flink, so the SR-Protobuf gap doesn't apply.

2.0 [OPTIONAL] AI Root-Cause Analysis (RCA)

Beyond the seven deterministic reports, an **eighth, AI-generated report** turns each stuck-trace *alert* into a natural-language root-cause hypothesis plus a one-line remediation. This **CCAF-only** feature is wired by `terraform/setup-ccaf-ai.tf` and can be enabled **at deploy time** by setting `ENABLE_TRACE_RCA=true` and supplying the other required `RCA` arguments to the `make cc-flink-reports-up` target.

When enabled, it adds three `confluent_flink_statement` resources:

- `CREATE MODEL trace_rca` — registers a remote text-generation model.
- A `proto-registry` sink table `isotope_report_trace_rca_1m`.
- An `INSERT … SELECT … LATERAL TABLE(ML_PREDICT('trace_rca', …))` that calls the model **once per alert** (reading the low-volume 1-minute stuck-trace report topic, never the source stream) and writes the hypothesis to its own topic — it never overwrites the deterministic reports.

The default provider is OpenAI (`gpt-4o`). Claude is supported via **AWS Bedrock** (`rca_model_provider = "bedrock"` with AWS credentials). Other supported providers: `vertexai`, `azureopenai`, `googleai`, `sagemaker`, `azureml`.

3.0 Layout

```
scripts/flink/sql/cp/                    CP Flink — session-cluster SQL
  00_source_table.fql                    CREATE TABLE per topic ('connector'
= 'kafka') + isotope_raw UNION view
  01_register_functions.fql              CREATE FUNCTION … USING JAR
'file:///opt/flink/lib/isotope-flink-udf.jar'
  05_isotope_view.fql                    Typed view; decodes x-isotope-*
header scalars (produces only)
  06_consume_events_view.fql             Typed view of
isotope_consume_edge_markers markers (consume edges)
  05_report_sinks.fql                    CREATE TABLE for each
```

```
isotope_report_*_1m Kafka sink (avro-confluent)
  10_latency_report.fql                 INSERT INTO: avg/min/max latency by
origin × topic
  20_topology_report.fql                INSERT INTO: produce-edge counts
per minute
  25_bipartite_topology_report.fql      INSERT INTO: produce plus consume
edges (bipartite graph) per minute
  30_hop_distribution.fql               INSERT INTO: hop-count buckets per
topic per minute
  40_coverage_report.fql                INSERT INTO: distinct traces per
topic per minute
  60_stuck_trace_report.fql             INSERT INTO: stuck-trace alerts via
STUCK_TRACE_PTF
  70_latency_percentiles_report.fql     INSERT INTO: p50/p95/p99 via
LATENCY_PERCENTILES PTF (T-Digest)
  07_flink_collector_sink.fql           CREATE TABLE orders.flink_enriched
- headers column declared
                                        WITHOUT VIRTUAL, which is what
makes it writable
  75_flink_collector.fql                INSERT INTO: Flink stamps its own
hop via ISOTOPE_APPEND_HOP
                                        (collector, not a report - 1:1, no
window)
  08_merge_provenance_sinks.fql         OPTIONAL - CREATE TABLE
orders.flink_batched (writable headers)
                                        + isotope_merge_edge_markers (typed
edge list)
  80_merge_collector.fql                OPTIONAL - INSERT INTO: windowed
merge, fresh trace via
                                        ISOTOPE_MERGE_TRACE (fan-in, so it
cannot continue a trace)
  81_merge_edge_markers.fql             OPTIONAL - one merge edge per
contributing trace (+ record count),
                                        labelled with the same derived ID
via ISOTOPE_MERGE_TRACE_ID
  82_merge_impact_query.fql             Ad-hoc SELECTs, never deployed:
stuck-trace alerts joined to
                                        the merge edges (run from `make
flink-sql`)
  09_state_provenance_sinks.fql         OPTIONAL - CREATE VIEW entity_log
(append-mode change log)
                                        + isotope_state_provenance (version
chain, parents inline)
  85_state_provenance.fql               OPTIONAL - INSERT INTO: one record
per emitted state via
                                        STATE_PROVENANCE (content-
addressed, no window, no hops)
  99_teardown.fql                       DROP TABLE/VIEW/FUNCTION (companion
to cp-flink-reports-down)
```

The two state-provenance files are applied only with `--state-provenance` (`STATE_PROVENANCE=true scripts/deploy-cmf-flink-reports.sh up`), and have **no CCAF equivalent** — see docs/state-

provenance.md. The three merge `OPTIONAL` files are applied only when the reports application is started with `--merge-provenance` (`MERGE_PROVENANCE=true scripts/deploy-cmf-flink-reports.sh up`); CCAF's equivalent is `terraform apply -var enable_merge_provenance=true`. Off, none of them is parsed and no extra topic is created. See docs/flink-collector.md §2.4.

CCAF runs the same seven reports plus the collector, but the SQL lives inline as `confluent_flink_statement` resources in terraform/setup-ccaf.tf — 24 statements: ALTER (×4) + raw view + typed produce view + typed consume view + 7 sinks + `STUCK_TRACE_PTF`, `LATENCY_PERCENTILES` and `ISOTOPE_APPEND_HOP` drop+register (×2 each = 6) + the collector sink and its two header ALTERs + **1 statement set holding all 8 INSERTs**. A normal apply creates 28, because the `ISOTOPE_MERGE_TRACE` / `ISOTOPE_MERGE_TRACE_ID` drop+register statements (×2 each = 4) live in terraform/setup-ccaf-merge-provenance.tf and are *not* gated — they register regardless of `var.enable_merge_provenance`, and only the 5 statements that *call* them are gated. The JAR is uploaded as a `confluent_flink_artifact` and referenced via `USING JAR 'confluent-artifact://<id>'`.

Why one statement rather than eight. CCAF's `EXECUTE STATEMENT SET BEGIN … END` runs multiple INSERTs as a single optimized statement, and Confluent documents it for exactly this case — INSERTs that read the same table or share intermediate results, which all eight do. It mirrors what CP already does through `IsotopeReportsJob`'s `StatementSet`, and it matters for cost: every separate CCAF statement carries its own 1-CFU compute-pool floor, so eight statements meant eight floors and a saturated pool. The tradeoff is one failure domain — a fault in any INSERT stops them all — which is the same property CP's single job has.

4.0 Wire-format detail (CP only)

Window columns ride as `BIGINT` epoch millis on the wire, not `TIMESTAMP_LTZ`. Flink 2.1.2's `avro-confluent` schema-derivation path raises `UnsupportedOperationException: Unsupported to derive Schema for type: TIMESTAMP_LTZ(3)` for that type. We cast in the INSERT (`UNIX_TIMESTAMP(CAST(window_start AS STRING)) * 1000`) so the on-wire schema is plain Avro `long`. Consumers rehydrate via `TO_TIMESTAMP_LTZ(window_start, 3)`. The CCAF Protobuf sinks don't hit this — `proto-registry` handles `TIMESTAMP_LTZ` directly.

5.0 PTF JAR

The single shadow JAR `ptf/build/libs/isotope-flink-udf.jar` (produced by `./gradlew :ptf:shadowJar`) carries all five JAR-backed functions under the `ai.signalroom.kafka.isotope.flink` package:

- `LatencyPercentilesPTF` (registered as `LATENCY_PERCENTILES`) — T-Digest p50/p95/p99 via per-window state + event-time timers.
- `StuckTracePTF` — per-trace state + event-time timer that emits alerts for traces idle ≥60s.
- `IsotopeAppendHop` (registered as `ISOTOPE_APPEND_HOP`) — a `ScalarFunction`, not a PTF. The two above are *interpreters*, reading headers an interceptor already wrote; this one is a *collector*, appending a Flink hop and rewriting the headers on the way out.

- `IsotopeMergeTrace` / `IsotopeMergeTraceId` (registered as `ISOTOPE_MERGE_TRACE` / `ISOTOPE_MERGE_TRACE_ID`) — the optional fan-in collector. `ISOTOPE_APPEND_HOP` continues an inbound trace, which is only truthful 1:1; these mint a **fresh** trace for a merged record and label its merge edges with the same deterministically derived ID. Registered on both runtimes regardless of the feature flag — registration is inert until a statement calls it.

All five register on both runtimes; only the `CREATE FUNCTION … USING JAR …` clause differs (`file://` path on CP, `confluent-artifact://` reference on CCAF).

One packaging trap. `Isotope` declares `fromHeaders(org.apache.kafka.common.header.Headers)`. `IsotopeAppendHop` never calls it, but the JVM resolves that descriptor when linking the class, so `Headers` must be present at *runtime*. `compileOnly` is not enough: CCAF's function classloader does not expose `kafka-clients`, and the statement fails on its first row — long after `CREATE FUNCTION` reported success — with `UDF invocation error: … org.apache.kafka.common.header.Headers`. The JAR therefore bundles `kafka-clients` (`transitive = false`), relocates `org.apache.kafka`, and ships only `common/header`: 2 classes rather than 4,152. `minimize()` cannot substitute — a type named only in a method descriptor is invisible to reachability analysis and gets stripped.

6.0 Operations

6.1 CP Flink on minikube

```
make cp-flink-up               # cert-manager → CFK Flink Operator → RustFS
→ CMF 2.5 → env → app image
make kafka-pf-up            # localhost:30092 → Kafka, localhost:8081 → SR
make cp-flink-reports-up    # build app JAR → upload as cmf:// artifact →
deploy the CMF Application
make cp-flink-reports-down  # delete the CMF Application + artifact + sink
topics
make cp-flink-down             # tear down the application, CMF, RustFS,
operator, cert-manager
```

The 7 reports run as a single Flink 2.1 CMF **Application** (`isotope-reports`, entry point `IsotopeReportsJob`) — visible in CMF and Control Center's Flink tab.

6.2 CCAF

```
make cc-flink-reports-up  CONFLUENT_API_KEY=... CONFLUENT_API_SECRET=...
                          # terraform apply: env + cluster + topics +
compute pool + artifact + 28 statements
                          # also regenerates terraform/terraform.png via
`terraform graph | dot`
```

or

```
make cc-flink-reports-up  CONFLUENT_API_KEY=... CONFLUENT_API_SECRET=...
ENABLE_TRACE_RCA=true RCA_MODEL_API_KEY=... RCA_MODEL_PROVIDER=...
RCA_MODEL_VERSION=... RCA_MODEL_ENDPOINT=...
                          # terraform apply: env + cluster + topics +
compute pool + artifact + 31 statements (28 + the 3 AI root-cause analysis
statements)
                          # also regenerates terraform/terraform.png via
`terraform graph | dot`
```

```
source scripts/cc-cli-env.sh          # exports BOOTSTRAP / SR_URL /
KAFKA_KEY / KAFKA_SECRET / SR_KEY / SR_SECRET / JAAS
scripts/cc-app-run.sh send orders.placed order-intake-service 'hello'   #
drives traffic with the SASL config
make cc-flink-reports-down CONFLUENT_API_KEY=... CONFLUENT_API_SECRET=...
                           # terraform destroy: deletes the environment and
everything in it
```

See the root README §3.3 "Flink SQL reports on Confluent Cloud for Apache Flink (CCAF)" for the full CCAF walkthrough, including the multi-window sustained-traffic pattern required to see tumbling-window aggregates emit.